# Week 2 Day 4 Compose App Lab

> Note: 이 폴더는 Day 5 Docker Compose 수업으로 넘어가기 위한 preview 자료다. Day 4 본 실습은 `docker run`, runtime config, logs, inspect, exec, stats, failure drill 중심으로 진행한다.

## Purpose

이 실습은 Day 3에서 길게 입력한 `docker run` 옵션을 `compose.yaml`로 옮긴다. 핵심은 파일 하나로 web, db, network, volume, environment, healthcheck, cleanup을 재현하는 것이다.

## Setup

```
cd week2/day4/labs/compose-app
cp .env.example .env
```

`.env`의 `POSTGRES_PASSWORD`는 로컬 실습용 값으로 바꾼다. 실제 운영 secret이나 개인 password를 쓰지 않는다.

## Run

```
docker compose config
docker compose up -d
docker compose ps
for i in 1 2 3 4 5; do
  curl -I http://localhost:18084 && break
  sleep 1
done
curl -s http://localhost:18084 | grep compose-site-v1
docker compose logs db
docker compose run --rm db-client
```

## 예상 확인 지점

```
web-1   running
db-1    healthy
HTTP/1.1 200 OK
compose-site-v1
```

```
current_database | current_user
paperclip        | paperclip
```

## Failure Drill

`.env`에서 `POSTGRES_PASSWORD` 줄을 제거한 뒤 config를 확인한다.

```
docker compose config
```

기대 결과는 `set POSTGRES_PASSWORD in .env` 메시지가 포함된 실패다.

## Cleanup

```
docker compose down
docker compose down -v
```

`docker compose down`은 container와 network를 정리한다. `docker compose down -v`는 named volume까지 삭제하므로 DB data가 사라진다.

## File Map

| Path | Purpose |
| --- | --- |
| `compose.yaml` | web, db, db-client service 정의 |
| `.env.example` | local `.env` 작성 기준 |
| `html/index.html` | nginx가 제공하는 정적 페이지 |
| `db/init/001_schema.sql` | PostgreSQL 최초 초기화 SQL |
| `README.md` | 실행, 확인, 실패, 정리 기준 |

## Services

| Service | Image | Role | Exposed To Host |
| --- | --- | --- | --- |
| `web` | `nginx:1.27-alpine` | static HTML server | yes, `${WEB_PORT:-18084}:80` |
| `db` | `postgres:16-alpine` | PostgreSQL database | no |
| `db-client` | `postgres:16-alpine` | query/check tool | no, profile tool |

`db-client`는 기본 application service가 아니라 확인용 service다. 필요할 때 `docker compose run --rm db-client`로 실행한다.

## Environment Variables

| Variable | Required | Default | Notes |
| --- | --- | --- | --- |
| `WEB_PORT` | no | `18084` | host에서 web에 접근할 port |
| `POSTGRES_DB` | no | `paperclip` | practice database name |
| `POSTGRES_USER` | no | `paperclip` | practice user |
| `POSTGRES_PASSWORD` | yes | none | `.env`에서 로컬 값으로 설정 |

`POSTGRES_PASSWORD`는 required variable이다. 값이 없으면 `docker compose config`가 실패한다. 의도적으로 실행 전 실패하게 만들어 secret 누락을 빨리 발견한다.

## Recommended Local Setup

```
cp .env.example .env
```

그 다음 `.env`를 열어 `POSTGRES_PASSWORD` 값을 로컬 실습용 값으로 바꾼다. 실제 개인 password, Docker Hub token, cloud access key를 쓰지 않는다.

## Preflight

```
docker version
docker compose version
docker compose config
```

Expected:

```
services:
  db:
  web:
networks:
  app-net:
volumes:
  pgdata:
```

# Detailed Runbook

## 1. Start

```
docker compose up -d
```

## 2. Check service status

```
docker compose ps
```

Expected signals:

```
web  Up
db   Up (healthy)
0.0.0.0:18084->80/tcp
```

## 3. Check web

```
curl -I http://localhost:18084
curl -s http://localhost:18084 | grep compose-site-v1
```

Expected:

```
HTTP/1.1 200 OK
compose-site-v1
```

## 4. Check DB

```
docker compose logs db
docker compose run --rm db-client
```

Expected:

```
database system is ready to accept connections
current_database | current_user
paperclip        | paperclip
```

# Network Model

The host reaches the web service through published port `localhost:18084`.

Containers inside the Compose network reach PostgreSQL through service name db.

| From | To | Address |
|---|---|---|
| host browser/curl | web | `localhost:18084` |
| `db-client` container | db | `db:5432` |
| host terminal | db | not published by default |

Do not document the DB container IP as the connection contract. Container IPs can change. Use service name `db`.

## Volume Model

`pgdata` stores PostgreSQL data at `/var/lib/postgresql/data`.

| Command | Result |
|---|---|
| `docker compose down` | removes containers and network |
| `docker compose down -v` | also removes named volumes |

Use `down -v` only when resetting practice data. Do not use it as the default cleanup for data that must be preserved.

## Failure Drill 1: Missing Env

Remove `POSTGRES_PASSWORD` from `.env`.

```
docker compose config
```

Expected failure:

```
set POSTGRES_PASSWORD in .env
```

Fix:

```
cp .env.example .env
```

Then set a local practice password again.

## Failure Drill 2: Wrong Port

```
curl -I http://localhost:80
curl -I http://localhost:18084
```

```
docker compose ps
```

Interpretation:

- `localhost:80` may fail because the published host port is `18084`.
- `localhost:18084` should return HTTP 200 when web is running.
- This is a host port issue, not an nginx container port issue.

## Failure Drill 3: Stale Volume

If you edit `db/init/001_schema.sql` after the database has already initialized, the change may not appear. PostgreSQL init scripts run only during initial database creation.

Practice reset:

```
docker compose down -v
docker compose up -d
```

Warning: this deletes practice DB data.

## Short RCA Template

```
## Compose App RCA
- Symptom:
- Failed command:
- Error excerpt:
- Category: config / port / env / network / volume / readiness
- Hypothesis:
- Fix:
- Recheck:
- Prevention:
```

## Security Notes

- Do not commit `.env`.
- Do not put real passwords or tokens in `.env.example`.
- Do not paste Docker Hub tokens, MFA codes, cloud access keys, or personal credentials into terminal screenshots.
- Prefer placeholder values in documentation.
- Keep DB port unpublished unless the exercise specifically requires host DB access.

# Handoff Section For Student README

```
## Docker Compose
### Setup
```

cp .env.example .env

```
### Run
```

docker compose config docker compose up -d

```
### Check
```

docker compose ps curl -I http://localhost:18084 curl -s http://localhost:18084 | grep compose-site-v1 docker compose run --rm db-client

```
### Cleanup
```

docker compose down

```
Reset practice DB data:
```

docker compose down -v

```
```

# 주의할 점 체크리스트

- [ ] `docker compose config` succeeded.
- [ ] `docker compose ps` shows web running.
- [ ] `docker compose ps` shows db running or healthy.
- [ ] `curl -I http://localhost:18084` returns HTTP 200.
- [ ] body contains `compose-site-v1`.
- [ ] `db-client` query returns `paperclip`.
- [ ] missing env failure is documented.
- [ ] cleanup command is documented.
- [ ] `down -v` data deletion warning is documented.

## Scoring

| Item | 0 | 1 | 2 |
|---|---|---|---|
| Config | not run | run only | output interpreted |
| Web | no check | status only | HTTP and body marker |
| DB | no check | healthy only | query 확인 지점 |
| Network | no explanation | network name only | service name `db` explained |
| Volume | no note | volume listed | data lifecycle explained |
| Security | secret exposed | warning only | `.env.example` and no secret exposure |
| Handoff | none | commands only | expected results and cleanup warning |

## Completion Statement

```
This Compose app is complete when another student can clone the files, create
`.env`, run `docker compose up -d`, verify web and DB 확인 지점, and clean up
without accidentally deleting data they intended to keep.
```